

MERGE: Next-Generation Item Indexing Paradigm for Large-Scale Streaming Recommendation

Jing Yan*
Bytedance
Singapore, Singapore
yanjing.rec@bytedance.com

Yimeng Bai
Bytedance
Shanghai, China
baiyimeng@bytedance.com

Zongyu Liu
Bytedance
Singapore, Singapore
liuzongyu@bytedance.com

Yahui Liu
Bytedance
Shanghai, China
liuyahui.lydia@bytedance.com

Junwei Wang
Bytedance
Shanghai, China
wangjunwei.00@bytedance.com

Jingze Huang
Bytedance
Shanghai, China
huangjingze@bytedance.com

Haoda Li
Bytedance
Singapore, Singapore
lihaoda@bytedance.com

Sihao Ding
Bytedance
Shanghai, China
dingsihao@bytedance.com

Shaohui Ruan
Bytedance
Shanghai, China
ruanshaohui@bytedance.com

Yang Zhang
National University of Singapore
Singapore, Singapore
zyang1580@gmail.com

Abstract

Item indexing, which maps a large corpus of items into compact discrete representations, is critical for both discriminative and generative recommender systems, yet existing Vector Quantization (VQ)-based approaches struggle with the highly skewed and non-stationary item distributions common in industrial streaming recommenders, leading to poor assignment accuracy, imbalanced cluster occupancy, and insufficient cluster separation. To address these challenges, we propose MERGE, a next-generation item indexing paradigm that adaptively constructs clusters from scratch, dynamically monitors cluster occupancy, and forms hierarchical index structures via fine-to-coarse merging. Extensive experiments demonstrate that MERGE significantly improves assignment accuracy, cluster uniformity, and cluster separation compared with existing indexing methods, while online A/B tests show substantial gains in key business metrics, highlighting its potential as a foundational indexing approach for large-scale recommendation. Codes are available at <https://github.com/baiyimeng/MERGE>.

CCS Concepts

• Information systems → Recommender systems.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
CIKM '26, Rome, Italy.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2539-5/2026/11
<https://doi.org/10.1145/3799682.3840083>

Keywords

Item Indexing; Item Tokenization; Large-Scale Streaming Recommendation

ACM Reference Format:

Jing Yan, Yimeng Bai, Zongyu Liu, Yahui Liu, Junwei Wang, Jingze Huang, Haoda Li, Sihao Ding, Shaohui Ruan, and Yang Zhang. 2026. MERGE: Next-Generation Item Indexing Paradigm for Large-Scale Streaming Recommendation. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26)*, November 7–11, 2026, Rome, Italy. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3799682.3840083>

1 Introduction

Item indexing, also referred to as item tokenization, which aims to map a large corpus of items into a compact, discrete set of indexes¹, plays a critical role in modern recommender systems [8, 9]. In traditional discriminative recommendation [35], it enables the construction of efficient index structures that can manage massive candidate sets while satisfying the low-latency requirements of the retrieval stage, thereby providing a solid foundation for the subsequent ranking stage [6, 34]. In emerging generative recommendation [7, 21], it supports the direct generation of items within a compressed token space, thus determining the upper bound of downstream recommender model performance [1, 15]. Consequently, item indexing has attracted substantial attention from both the academic and industrial communities [2, 23, 25].

¹The terms “indexes”, “tokens”, “codes”, “codewords”, and “clusters” are often used interchangeably in different contexts.

Early works on item indexing explored handcrafted tree-based [33, 34, 36] and hierarchical path [6] index structures, achieving a breakthrough beyond classical two-tower retrieval frameworks. Motivated by the remarkable success of generative AI [14, 28], recent studies have increasingly adopted Vector Quantization (VQ)-based tokenization techniques [24] for recommendation [2, 3, 18, 21, 26, 27, 29, 32], which learn one or more codebooks that serve as the basis for item indexing. For instance, StreamingVQ [2] directly employs VQ for single-layer clustering to enhance index immediacy, effectively replacing all major online retrievers. Trinity [27] utilizes VQ for two-level clustering to construct statistical interest histograms of users, providing a unified solution for modeling multi-interest, long-tail, and long-term interests.

Despite their widespread success, we argue that VQ techniques are poorly suited for industrial streaming recommendation scenarios. When faced with highly skewed and rapidly evolving item distributions [30], VQ methods encounter several challenges. Specifically, focusing on a single-layer VQ, *i.e.*, a single codebook comprising multiple codewords (clusters), we identify three key challenges: (1) *Accuracy*: Item embeddings are poorly aligned with their assigned cluster embeddings, with an average cosine similarity of only 0.6, potentially leading to reduced retrieval precision. (2) *Uniformity*: Cluster occupancy is highly imbalanced, with some clusters containing orders of magnitude more items than others. This imbalance typically manifests in the indiscriminate grouping of long-tail items, thereby hindering the effective retrieval of high-quality long-tail candidates. (3) *Separation*: Cluster embeddings are insufficiently distinct, exhibiting an average inter-cluster cosine similarity exceeding 0.5, which can result in training instability, *i.e.*, the same item being assigned to different clusters over short periods. These challenges could be further exacerbated by feedback loops [11], ultimately destabilizing the recommendation ecosystem.

This naturally raises a broader question: can we establish a new solution for item indexing, specifically designed for industrial recommendation, that overcomes these inherent limitations? Addressing these challenges calls for a rethinking of the fundamental principles of index construction. Regarding accuracy, when assigning an item to a cluster, the assignment should be rejected if the similarity is too low, and new clusters should be dynamically created when necessary. In terms of uniformity, adaptive mechanisms are needed to regulate cluster sizes within reasonable bounds, thereby mitigating severe occupancy imbalances. Regarding separation, cluster construction should be context-aware, taking the existence of other clusters into account to ensure adequate distinctiveness.

In response, we propose **MERGE**, a next-generation item indexing paradigm tailored for large-scale streaming recommendation. Unlike VQ, which depends on a predefined number of clusters, MERGE adaptively generates clusters entirely from scratch, thereby enabling the formation of a dynamic codebook that evolves with the streaming data. Within each updating step, items are initially matched to existing clusters based on similarity, with matches exceeding a threshold updating cluster embeddings via the Exponential Moving Average (EMA) algorithm [13]. Items that fail to match are processed using a Union-Find algorithm [12] to form new clusters, while those insufficient to establish a cluster center are temporarily cached. Cluster occupancy is also dynamically monitored, retaining only clusters with an appropriate number of items

and resetting those that do not meet this criterion. Overall, this approach—combining dynamic cluster construction with real-time occupancy monitoring—overcomes the aforementioned limitations. Building on this, we treat these clusters as a fine-grained codebook and adaptively aggregate it into coarser-grained codebooks, ultimately forming a hierarchical indexing structure. We conduct both offline and online evaluations, yielding solid evidence that attests to its superior recommendation performance.

The main contributions of this work are summarized as follows:

- We identify three fundamental challenges in existing item indexing techniques for large-scale streaming recommendation—accuracy, uniformity, and separation—highlighting the limitations of VQ-based approaches in handling highly skewed and non-stationary item distributions.
- We propose MERGE, a next-generation item indexing paradigm that adaptively generates clusters from scratch, dynamically monitors cluster occupancy, and builds a hierarchical indexing structure via fine-to-coarse merging, providing a principled solution tailored for large-scale recommendation.
- We conduct extensive offline evaluations and online deployments, demonstrating that MERGE consistently outperforms baseline indexing methods and yields significant improvements across multiple business metrics.

2 Preliminaries

2.1 Task Formulation

Item indexing maps items into compact discrete representations, enabling efficient index structures for retrieval in multi-stage recommendation pipelines [6, 34]. These representations can also serve as token spaces for generative recommendation, where recommendation is formulated as sequence generation [32].

Each item i is represented by an embedding $\mathbf{e}_i$, obtained from either a pretrained recommender model [17] or a multimodal model such as Qwen [28]. Item indexing learns a mapping:

$$f : \mathbf{e}_i \rightarrow [c^1, c^2, \dots, c^L]. \quad (1)$$

Here, f assigns $\mathbf{e}_i$ a discrete index of L codewords. In traditional retrieval, L is usually 1 or 2, allowing multiple items to share an index [2, 27]. Generative recommenders often use longer indices (*e.g.*, $L = 3$ or 4), with an additional unique code to ensure one-to-one item-index correspondence [17, 21].

2.2 VQ-Based Indexing Paradigm

We first consider single-layer VQ and omit VAE-based extensions for simplicity [24]. In this case, $L = 1$, and the discrete representation space is a learnable codebook $\mathcal{C} = \{\mathbf{c}_k\}_{k=1}^K$, where K is the codebook size and each codeword $\mathbf{c}_k$ has the same dimension as $\mathbf{e}_i$. Each item is assigned to the nearest codeword under a similarity metric $\text{sim}(\cdot, \cdot)$, typically the negative squared Euclidean distance:

$$c^1 = \arg \max_{k \in \{1, \dots, K\}} \text{sim}(\mathbf{e}_i, \mathbf{c}_k). \quad (2)$$

This formulation extends naturally to multi-layer residual quantization (RQ) [21], which uses coarse-to-fine residual dependencies with codebooks $\mathcal{C}^l = \{\mathbf{c}_k^l\}_{k=1}^K$, $l = 1, \dots, L$. The l -th quantization

layer is defined as:

$$\begin{aligned} c^l &= \arg \max_{k \in \{1, \dots, K\}} \text{sim}(\mathbf{r}^l, \mathbf{c}_k^l), \\ \mathbf{r}^l &= \mathbf{r}^{l-1} - \mathbf{c}_{c^l}^l, \\ \mathbf{r}^0 &= \mathbf{e}_i, \end{aligned} \quad (3)$$

where $\mathbf{c}_{c^l}^l$ is the selected codeword from the l -th codebook, $\mathbf{r}^l$ is the residual after layer l , and $\mathbf{r}^0$ is the original item embedding.

3 Methodology

3.1 Overview

We propose **MERGE**, an item indexing method for industrial streaming recommendation, to address the three challenges in Section 1: *accuracy*, *uniformity*, and *separation*. As shown in Figure 1, MERGE dynamically organizes incoming items into clusters while keeping them balanced and well-separated.

MERGE integrates three mechanisms: (1) dynamic cluster construction, which creates and updates clusters as new items arrive; (2) real-time occupancy monitoring, which balances cluster sizes and mitigates skew—together forming a single-layer codebook; and (3) fine-to-coarse merging, which builds a hierarchical index for complex business requirements. These components allow MERGE to adapt to evolving item distributions in streaming scenarios.

3.2 MERGE Framework

3.2.1 Dynamic Cluster Construction. This component incrementally updates clusters from streaming item data, forming a dynamic codebook. We describe the batch-wise update procedure below.

At each step, a batch of items $\mathcal{B}$ is sampled from the item corpus. Each item is represented by a *64-dimensional collaborative embedding* produced by a retriever model serving billions of users. We maintain an initially empty codebook $\mathcal{Q} = \{\mathbf{q}_k\}_{k=1}^K$, where each codeword $\mathbf{q}_k$ represents a cluster and K evolves over time. Codewords are updated by Exponential Moving Average (EMA) [13]. For each $\mathbf{q}_k$, we track the EMA sum of assigned item embeddings S_k and the EMA count N_k .

Step 1: Similarity Computation. For each item embedding $\mathbf{e}_i \in \mathcal{B}$, we compute its similarity to all existing codewords and identify the best match:

$$\begin{aligned} k_i^* &= \arg \max_{k \in \{1, \dots, K\}} \text{sim}(\mathbf{e}_i, \mathbf{q}_k), \\ s_i^* &= \max_{k \in \{1, \dots, K\}} \text{sim}(\mathbf{e}_i, \mathbf{q}_k). \end{aligned} \quad (4)$$

We use cosine similarity, which performs well empirically and yields values in $[-1, 1]$ for threshold-based assignment. When embeddings are ℓ_2 -normalized, maximizing cosine similarity is equivalent to minimizing the squared Euclidean distance.

Step 2: Matching Decision. Given a threshold $\tau \in [0, 1]$, items are partitioned into matched and unmatched sets:

$$\begin{aligned} \mathcal{B}^+ &= \{i \in \mathcal{B} \mid s_i^* \geq \tau\}, \\ \mathcal{B}^- &= \{i \in \mathcal{B} \mid s_i^* < \tau\}, \end{aligned} \quad (5)$$

where $\mathcal{B}^+$ and $\mathcal{B}^-$ denote successfully matched and failed items, respectively.

Step 3: Cluster Update. For each codeword $\mathbf{q}_k$, we collect its matched items:

$$\mathcal{B}_k^+ = \{i \in \mathcal{B}^+ \mid k_i^* = k\}. \quad (6)$$

Then we update its EMA variables:

$$\begin{aligned} S_k &\leftarrow \gamma S_k + (1 - \gamma) \sum_{i \in \mathcal{B}_k^+} \mathbf{e}_i, \\ N_k &\leftarrow \gamma N_k + (1 - \gamma) |\mathcal{B}_k^+|, \end{aligned} \quad (7)$$

where γ is typically set to 0.99. The codeword is then updated as:

$$\mathbf{q}_k \leftarrow \frac{S_k}{N_k}. \quad (8)$$

This update enables codewords to adapt to new items while maintaining stability, improving item-to-cluster assignment accuracy.

Step 4: Cluster Extension. For unmatched items $\mathcal{B}^-$, we construct new clusters using Union-Find [12]. This procedure forms connected components under a pairwise similarity threshold without requiring a fixed number of clusters or iterative centroid optimization [4, 22].

Specifically, we compute pairwise similarities in $\mathcal{B}^-$, connect item pairs above a threshold, and merge connected items into disjoint clusters:

$$\mathcal{U} = \text{UF}(\mathcal{B}^-, \tau'), \quad (9)$$

where $\mathcal{U}$ is the set of cluster embeddings obtained by mean-pooling items in each cluster, and $\tau' \in [0, 1]$ is the connection threshold.

We retain only clusters whose sizes exceed a threshold m :

$$\mathcal{U}^{\text{valid}} = \{\mathbf{u} \in \mathcal{U} \mid \text{size}(\mathbf{u}) \geq m\}, \quad (10)$$

where $\text{size}(\mathbf{u})$ is the number of items in cluster $\mathbf{u}$.

Let $\mathcal{Q}^{\text{zero}} \subseteq \mathcal{Q}$ be the reset, zeroed cluster embeddings. To incorporate $\mathcal{U}^{\text{valid}}$, we fill reset entries first and append the rest via the *Fill-Then-Append (FTA)* operator:

$$\mathcal{Q} \leftarrow \text{FTA}(\mathcal{Q}, \mathcal{U}^{\text{valid}}). \quad (11)$$

Filling zeroed clusters preserves codebook order and improves stability. The EMA variables for new clusters are initialized accordingly. Since $\mathcal{U}^{\text{valid}}$ comes from unmatched items, the new clusters are naturally distinct from existing codewords, enhancing separation and reducing redundancy.

Step 5: Item Recycling. Clusters in $\mathcal{U} \setminus \mathcal{U}^{\text{valid}}$ fail both matching and valid cluster formation. Their items are recycled into the data queue and await future batches for subsequent updates.

Tag-Based Batch Formation. In streaming settings, item embeddings within the same batch may be highly diverse, causing noisy and unstable cluster updates. To improve homogeneity and accelerate convergence, we use prior tags from multimodal information, covering 100 categories, and construct each batch with items sharing the same tag. This strategy mainly optimizes training when the batch size is not sufficiently large, leading to faster convergence and more stable early-stage cluster formation; it can be removed when the batch size is sufficiently large.

3.2.2 Real-Time Occupancy Monitoring. This component runs alongside cluster updates to maintain balanced cluster sizes. Each cluster

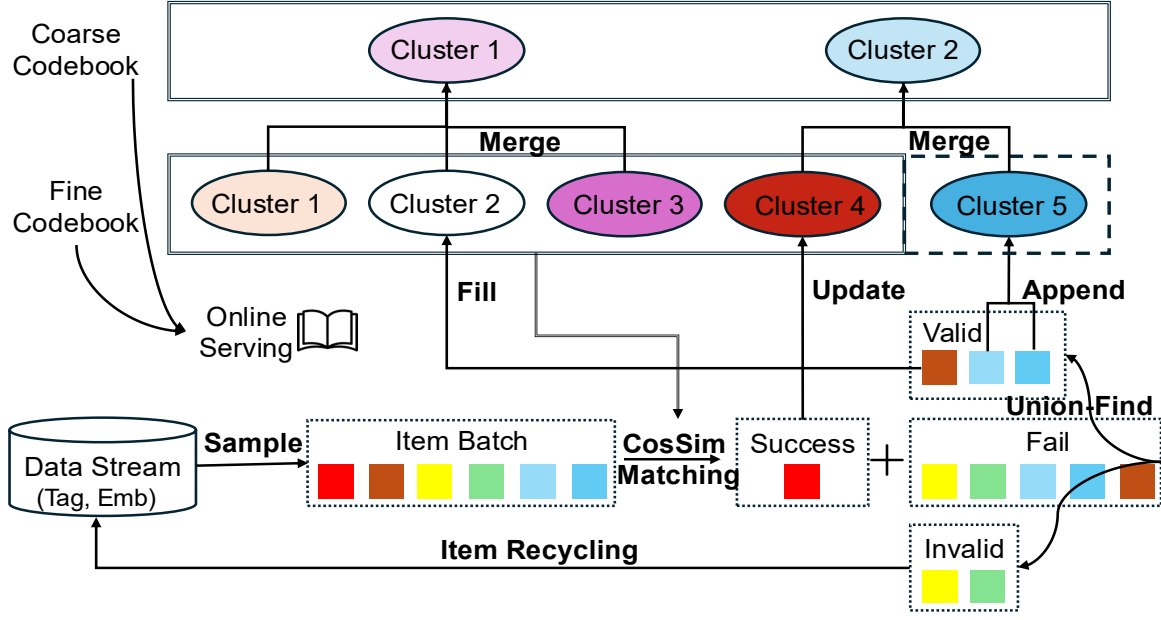


Figure 1: Overview of MERGE. Incoming item batches are matched to the codebook via cosine similarity. Matched items update existing clusters, while unmatched items form new clusters through union-find. Valid new clusters are filled or appended, and the rest are recycled. A fine-to-coarse merging process further builds a hierarchical codebook for efficient indexing.

q_k tracks its EMA count N_k , and is categorized by two thresholds ε_1 and ε_2 :

$$\text{status}(q_k) := \begin{cases} \text{underfilled,} & N_k < \varepsilon_1, \\ \text{growing,} & \varepsilon_1 \leq N_k < \varepsilon_2, \\ \text{stable,} & N_k \geq \varepsilon_2. \end{cases} \quad (12)$$

Stable clusters remain unchanged. Underfilled clusters are *reset*: their embeddings and EMA variables are zeroed, and associated items are cleared, forming $Q^{\text{zero}} \subseteq Q$. Growing clusters are monitored over M steps and reset if they fail to become stable. Adjusting ε_1 and ε_2 helps balance cluster occupancy, avoiding dominant large clusters and redundant small ones.

3.2.3 Fine-to-Coarse Merging. Given the fine-grained codebook Q , we construct a hierarchical codebook by building a coarser codebook $\mathcal{P}$ on top. Initially, $\mathcal{P} = Q$.

Step 1: Merging. We define the *affinity score* between p_x and p_y in $\mathcal{P}$ as:

$$w(p_x, p_y) = \text{sim}(p_x, p_y) - \lambda \min(N_x, N_y), \quad (13)$$

where $\text{sim}(\cdot, \cdot)$ is cosine similarity, and N_x, N_y are EMA counts. The second term penalizes high-occupancy clusters, encouraging more uniform merging.

At each iteration, we merge the pair with the highest affinity. The merged embedding is computed by count-weighted averaging:

$$p_{xy} = \frac{N_x p_x + N_y p_y}{N_x + N_y}, \quad (14)$$

with count updated accordingly. The two prototypes are replaced by p_{xy} , and affinities are recomputed until the target cluster number is reached.

After merging, $\mathcal{P}$ contains merged clusters $\mathcal{P}^{\text{mrg}}$ and unmerged prototypes $\mathcal{P}^{\text{umrg}}$.

Step 2: Pruning. For each merged cluster $p \in \mathcal{P}^{\text{mrg}}$, we assess each prototype q using the *silhouette coefficient* [5]:

$$r_q = \frac{b_q - a_q}{\max\{a_q, b_q\}}, \quad (15)$$

where a_q is the average distance to prototypes in the same merged cluster, and b_q is the minimum average distance to prototypes in other clusters. The distance is defined as $1 - w(p_x, p_y)$. Prototypes with scores below threshold r are pruned.

Step 3: Reconnection. The pruned prototypes $\mathcal{P}^{\text{prun}}$ and unmerged prototypes $\mathcal{P}^{\text{umrg}}$ are reintroduced into the merging procedure. By iteratively applying merging, pruning, and reconnection, we obtain the desired coarse-grained codebook $\mathcal{P}$.

These steps can be recursively applied for even coarser layers. In this work, we construct only $\mathcal{P}$, which satisfies our online deployment requirements.

4 Experiment

RQ1: How does MERGE compare with baseline indexing methods in accuracy, uniformity, and separation?

RQ2: Can MERGE's indexing improvements translate into measurable gains in user satisfaction and business metrics?

RQ3: How do the item-cluster assignments produced by MERGE and baselines differ qualitatively?

4.1 Offline Evaluation (RQ1)

We first evaluate MERGE with respect to the three challenges discussed above.

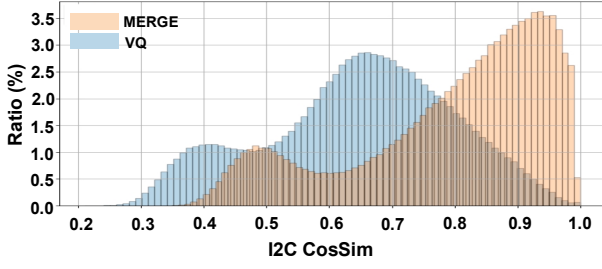


Figure 2: Accuracy comparison between MERGE and VQ, illustrating the item-to-cluster cosine similarity (I2C CosSim) distributions within the codebook.

Dataset. Since MERGE targets large-scale streaming recommendation, we train and evaluate it on platform-specific industrial data rather than public datasets. Following StreamingVQ [2], MERGE is trained on a candidate stream where candidates are presented sequentially with equal probability, which is more suitable for indexing than popularity-biased impression streams [25]. The data contains hundreds of millions of candidates, providing sufficient coverage and diversity.

Implementation Details. We use a retriever model to provide real-time 64-dimensional item embeddings, incorporating item ID, author features, statistical signals, and content-aware attributes. MERGE is trained with batch size 20,480 to ensure stable cluster assignment. After convergence, the fine-grained codebook contains tens of thousands of valid clusters. Training outputs are written back for subsequent updates. MERGE involves only a few intuitive hyper-parameters, such as assignment/merging thresholds, EMA smoothing, cluster-size regularization, and occupancy-based filtering. *In our experiments, performance is robust to moderate variations of these choices, and detailed sensitivity studies are omitted due to space constraints.*

Baseline. We compare MERGE with StreamingVQ [2], denoted as VQ, which uses a predefined-size single codebook and represents a canonical VQ-based indexing paradigm validated at large scale.

Item Sampling. We randomly sample items $\mathcal{D}$ from the stream and obtain their assigned cluster embeddings:

$$\{(i, \mathbf{q}_i) \mid i \in \mathcal{D}\}.$$

Here, $\mathbf{q}_i$ denotes the corresponding cluster embedding. For MERGE, it is taken from the fine-grained codebook $\mathcal{Q}$. We then evaluate accuracy, uniformity, and separation as follows.

4.1.1 Accuracy. We evaluate whether each item is close to its assigned cluster by computing item-to-cluster cosine similarity (I2C CosSim). As shown in Figure 2, VQ has a mean similarity around 0.6, while MERGE reaches about 0.9. This indicates that MERGE produces more semantically coherent assignments, mainly because it performs threshold-based matching rather than blind assignment.

4.1.2 Uniformity. We examine cluster sizes, i.e., the number of items assigned to each cluster. For finer analysis, videos are grouped by Video Views (VV), and we visualize how videos in each VV bucket are distributed across clusters. As shown in Figure 3, the largest MERGE cluster contains about 17,500 items, while the largest

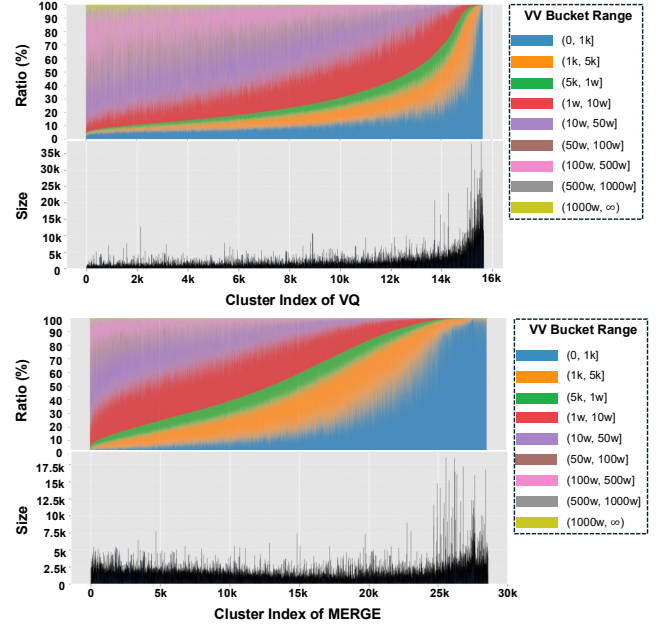


Figure 3: Uniformity comparison between MERGE and VQ, showing cluster sizes and the cumulative distribution of videos within the VV bucket range across clusters.

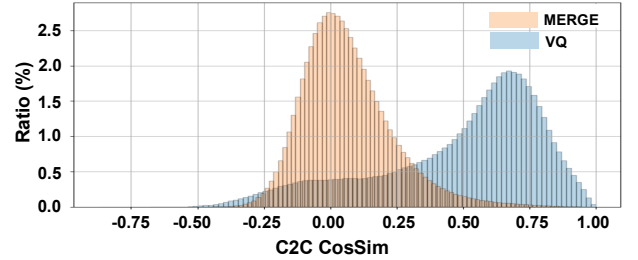


Figure 4: Separation comparison between MERGE and VQ, illustrating the cluster-to-cluster cosine similarity (C2C CosSim) distributions within the codebook.

VQ cluster reaches around 40,000 items, indicating better overall uniformity. Moreover, VQ tends to group low-VV videos together and leaves most clusters dominated by high-popularity items. In contrast, MERGE distributes low-VV videos more evenly, mitigating popularity dominance. This validates the effectiveness of occupancy monitoring and reset operations.

4.1.3 Separation. We measure pairwise cluster similarity using cluster-to-cluster cosine similarity (C2C CosSim). As shown in Figure 4, MERGE produces an approximately normal distribution centered near zero, whereas VQ has a much higher mean similarity around 0.6. This demonstrates stronger cluster separation in MERGE, mainly due to its dynamic codebook expansion. Better

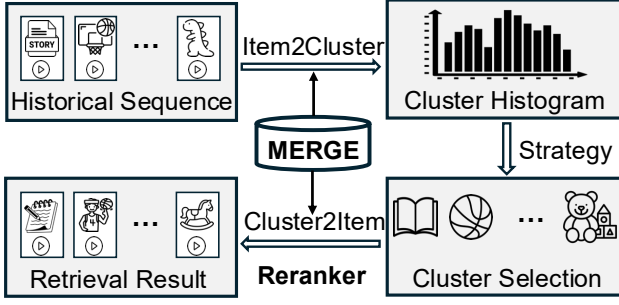


Figure 5: Overview of MERGE deployment during serving, following the Trinity [27] pipeline. MERGE provides a bidirectional mapping between clusters and items.

Table 1: Relative changes in core metrics observed during the A/B test. AAD and AAH denote Average Active Days and Average Active Hours, respectively.

Aspect	Metric	Relative Change
Engagement	AAD	+0.0081%
	AAH	+0.0546%
	WatchTime	+0.1006%
VV Stratification	(0,1k]	+1.49%
	(1k,5k]	+11.07%
	(5k, 10k]	+11.87%
	(10k, 100k]	+2.25%
	(100k, ∞)	-2.65%
Freshness Stratification	(0, 2h]	+0.59%
	(2h, 12h]	+7.64%
	(12h, 24h]	-0.29%
Diversity	(24h, 72h]	-1.88%
	TagNum	+0.22%
	TagGini	-0.03%

separation also improves training stability by reducing abrupt item-cluster assignment changes, with theoretical justification provided in Appendix A.

4.2 Online Deployment (RQ2)

After validating MERGE offline, we deploy it in our online recommender system to test whether indexing improvements lead to user satisfaction gains.

Pipeline. We follow Trinity [27], which uses VQ-based two-level clustering to build statistical interest histograms for modeling multi-interest, long-tail, and long-term user interests. We replace its original VQ indexing with MERGE. As shown in Figure 5, for each request, we extract the user’s long-term behavior sequence and build a histogram over clusters using item–cluster mappings. Clusters are then selected by a predefined strategy, such as top-ranked clusters, following Trinity. Candidate items are retrieved through cluster–item mappings and fed into a reranker, followed by pre-ranking, ranking, and other downstream stages [6].

A/B Test. We conduct a one-week A/B test with millions of users. The control group uses Trinity with VQ, while the treatment

Table 2: Relative changes in single-path metrics observed during the A/B test.

Aspect	Metric	Relative Change
Survivability	Pass-Through Rate	+45.04%
Contribution	Output Ratio	+85.99%
	Unique Output Ratio	+26.36%
	Staytime	+9.84%
Action	Like	+8.78%
	Follow	+17.32%
	Share	+7.65%
	Comment	+39.43%

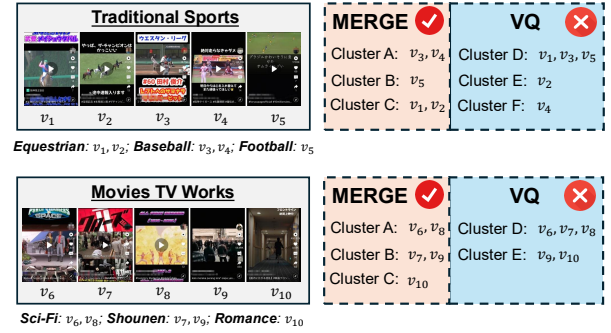


Figure 6: Case study comparing the cluster assignments of retrieved videos under MERGE and VQ.

group uses Trinity with MERGE. Raw counts and exact traffic allocation are commercially sensitive, so we report aggregated relative changes.

4.2.1 Core Metric Analysis. As an entertainment platform, our primary goal is to improve user engagement. Since DAU is difficult to observe directly under balanced A/B assignment, we follow Trinity and use Average Active Days (AAD), Average Active Hours (AAH), and Watch Time as proxy metrics. We further analyze content composition and diversity. VV stratification measures exposure across popularity levels, freshness stratification measures exposure across publication-time ranges, and diversity is evaluated by TagNum and TagGini. As shown in Table 1, MERGE improves all engagement metrics, with statistically significant gains at the 5% level. Although the relative gains are modest, they imply substantial practical impact at billion-user scale. Stratification results show that MERGE increases exposure of low-VV and recently published content, while also slightly improving diversity.

These gains stem from three factors. First, improved accuracy assigns items to more semantically appropriate clusters, increasing retrieval relevance. Second, better uniformity balances cluster sizes and improves the visibility of low-VV items. Third, stronger separation reduces over-concentration in popular semantic regions, promoting freshness and diversity. Together, these indexing-level improvements propagate through the recommendation pipeline and enhance user engagement.

4.2.2 Single-Path Metric Analysis. We evaluate MERGE along a single Trinity retrieval path from three dimensions: survivability, contribution, and action. Survivability measures whether retrieved items survive later ranking stages, corresponding to Pass-Through Rate. Contribution measures the share of final impressions from this path, before and after deduplication. Action measures user feedback, including Staytime, Like, Follow, Share, and Comment. As shown in Table 2, MERGE improves all single-path metrics. Retrieved items are more likely to pass through downstream stages, contribute more and more distinct final impressions, and trigger stronger user interactions. These results indicate that single-path retrieval improvements support the overall online gains.

4.2.3 Efficiency Analysis. MERGE is significantly more resource-efficient than VQ because it operates on the item corpus rather than massive user interaction logs. In our online setting, VQ requires a distributed cluster of 1,000 instances with 4 CPU cores each, whereas MERGE achieves superior indexing quality on a single 30-core instance. This greatly reduces synchronization overhead and infrastructure costs.

4.3 Case Study (RQ3)

We present a real-world case study of item-cluster mappings. For each request, we retrieve about 1,000 candidate videos and use tag-based filtering to retain around 100 videos of the same theme. We then rank the assigned clusters by frequency and inspect their videos. The selected themes include “*Traditional Sports*” and “*Movies TV Works*”. For a fair comparison, both methods use codebooks of approximately 20,000 clusters. As shown in Figure 6, MERGE produces fewer and more semantically coherent clusters, whereas VQ yields fragmented and noisy assignments. For *Traditional Sports*, MERGE groups baseball videos (v_3, v_4) and equestrian videos (v_1, v_2) into their respective clusters. For *Movies TV Works*, it similarly groups Sci-Fi videos (v_6, v_8) and Shounen videos (v_7, v_9). In contrast, VQ scatters related videos across clusters and introduces cross-category contamination. These cases demonstrate MERGE’s stronger discriminability and assignment accuracy, stemming from its paradigm shift from matching clusters to generating clusters. Accuracy aligns items with proper clusters, while uniformity and separation preserve fine-grained partitioning. Even when enlarging the VQ codebook, these issues remain, suggesting that the limitation lies in its underlying mechanism rather than scale.

5 Related Work

Efficient item indexing is essential for large-scale recommendation, as exhaustive corpus scanning is infeasible. Classical ANN methods such as Product Quantization (PQ [10]), Navigable Small World (NSW [19]), and Hierarchical NSW (HNSW [20]) enable fast approximate retrieval through quantization or graph-based search. Tree-based methods, including TDM [34] and JTM [33], organize items hierarchically for coarse-to-fine candidate search, while Deep Retrieval [6] avoids the Euclidean-space assumption of typical ANN methods and performs efficient path-based indexing. Recently, StreamingVQ [2] applies VQ to streaming item indexing and achieves notable online gains. StreamingVQ is the closest baseline because it consumes streaming item representations and uses the same one-codebook interface. In contrast, MERGE establishes

a cluster-generation paradigm for indexing, with both theoretical and empirical evidence showing its advantages over traditional VQ-based methods.

In generative recommendation [7], item indexing is often studied as item tokenization. TIGER [21] uses RQ-VAE to quantize item text embeddings into token sequences, reformulating recommendation as sequence generation. OneRec [32] adopts a similar strategy and deploys RQ-KMeans at scale in Kuaishou. Most existing methods [1, 3, 9, 17, 18, 25, 26, 29] still rely on VQ, whose inherent paradigm limitations make them less suitable for large-scale streaming recommendation.

6 Conclusion

In this work, we investigated the limitations of existing item indexing methods and identified three challenges in industrial streaming recommendation: poor assignment accuracy, imbalanced cluster occupancy, and insufficient cluster separation. We proposed MERGE, an item indexing framework that constructs clusters from scratch, dynamically monitors occupancy, and builds a hierarchy via fine-to-coarse merging. MERGE provides a promising indexing paradigm for large-scale streaming recommendation.

MERGE still has limitations. Low-VV videos remain challenging, as shown in Figure 3. The framework is also more complex than VQ-based methods, and its current deployment is limited to traditional retrieval. Future work will explore multimodal item embeddings, more efficient hierarchical construction, and the extension of MERGE to generative retrieval and reasoning [16, 31].

A Theoretical Stability Analysis

Let $\mathbf{Q} = \{\mathbf{q}_k\}_{k=1}^K$ be a codebook with normalized codewords, *i.e.*, $\|\mathbf{q}_k\| = 1$. Suppose item $\mathbf{e}_i$ is assigned to $\mathbf{q}_k$, such that

$$\mathbf{q}_k^\top \mathbf{e}_i \geq \mathbf{q}_j^\top \mathbf{e}_i, \quad \forall j \neq k. \quad (16)$$

Under a perturbation δ with $\|\delta\| \leq \epsilon$, the assignment is preserved if

$$(\mathbf{q}_k - \mathbf{q}_j)^\top \mathbf{e}_i \geq (\mathbf{q}_j - \mathbf{q}_k)^\top \delta, \quad \forall j \neq k. \quad (17)$$

By Cauchy–Schwarz, a sufficient condition is

$$(\mathbf{q}_k - \mathbf{q}_j)^\top \mathbf{e}_i \geq \epsilon \sqrt{2(1 - \mathbf{q}_k^\top \mathbf{q}_j)}, \quad \forall j \neq k, \quad (18)$$

where we use $\|\mathbf{q}_k - \mathbf{q}_j\|^2 = 2(1 - \mathbf{q}_k^\top \mathbf{q}_j)$.

To relate stability to codeword separation, decompose

$$\mathbf{e}_i = \alpha \mathbf{q}_k + \Delta, \quad \alpha = \mathbf{q}_k^\top \mathbf{e}_i > 0, \quad \mathbf{q}_k^\top \Delta = 0. \quad (19)$$

Then,

$$(\mathbf{q}_k - \mathbf{q}_j)^\top \mathbf{e}_i = \alpha(1 - \mathbf{q}_k^\top \mathbf{q}_j) - \mathbf{q}_j^\top \Delta \geq \alpha(1 - \mathbf{q}_k^\top \mathbf{q}_j) - \|\Delta\|. \quad (20)$$

Consequently, Equation (18) holds if

$$\alpha(1 - \mathbf{q}_k^\top \mathbf{q}_j) - \|\Delta\| \geq \epsilon \sqrt{2(1 - \mathbf{q}_k^\top \mathbf{q}_j)}, \quad (21)$$

which is equivalent to

$$\mathbf{q}_k^\top \mathbf{q}_j \leq 1 - \frac{(\epsilon + \sqrt{\epsilon^2 + 2\alpha\|\Delta\|})^2}{2\alpha^2}. \quad (22)$$

Therefore, greater codeword separation provides a larger stability margin against bounded perturbations, supporting the empirical observation in Figure 4.

GenAI Usage Disclosure

The authors used generative AI tools for language polishing, grammar correction, and improving the clarity and conciseness of the manuscript. All technical content, experiments, analyses, and conclusions were developed and verified by the authors, who have reviewed the resulting text and take full responsibility for the final manuscript.

References

- [1] Yimeng Bai, Chang Liu, Yang Zhang, Dingxian Wang, Frank Yang, Andrew Rabinovich, Wenge Rong, and Fuli Feng. 2026. Bi-Level Optimization for Generative Recommendation: Bridging Tokenization and Generation. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval* (SIGIR '26). Association for Computing Machinery, New York, NY, USA, 53–64.
- [2] Xingyan Bin, Jianfei Cui, Wujie Yan, Zhichen Zhao, Xintian Han, Chongyang Yan, Feng Zhang, Xun Zhou, Xiao Yang, and Zuotao Liu. 2025. Real-time Indexing for Large-scale Recommendation by Streaming Vector Quantization Retriever. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Toronto ON, Canada) (KDD '25). Association for Computing Machinery, New York, NY, USA, 4273–4283.
- [3] Ben Chen, Xian Guo, Siyuan Wang, Zihan Liang, Yufei Ma, Yue Lv, Chenyi Lei, Yuqing Ding, Wenwu Ou, Han Li, and Kun Gai. 2026. OneSearch: A Preliminary Exploration of the Unified End-to-End Generative Framework for E-commerce Search. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. PMLR.
- [4] Laxman Dhulipala, Xiaojun Dong, Kishen N. Gowda, and Yan Gu. 2024. Optimal Parallel Algorithms for Dendrogram Computation and Single-Linkage Clustering. In *Proceedings of the 36th ACM Symposium on Parallelism in Algorithms and Architectures* (Nantes, France) (SPAA '24). Association for Computing Machinery, New York, NY, USA, 233–245.
- [5] Duy-Tai Dinh, Tsutomu Fujinami, and Van-Nam Huynh. 2019. Estimating the optimal number of clusters in categorical data clustering by silhouette coefficient. In *International Symposium on Knowledge and Systems Sciences*. Springer, 1–17.
- [6] Weihao Gao, Xiangjun Fan, Chong Wang, Jiankai Sun, Kai Jia, Wenzhi Xiao, Ruofan Ding, Xingyan Bin, Hui Yang, and Xiaobing Liu. 2021. Learning An End-to-End Structure for Retrieval in Large-Scale Recommendations. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management* (Virtual Event, Queensland, Australia) (CIKM '21). Association for Computing Machinery, New York, NY, USA, 524–533.
- [7] Yupeng Hou, An Zhang, Leheng Sheng, Zhengyi Yang, Xiang Wang, Tat-Seng Chua, and Julian McAuley. 2025. Generative Recommendation Models: Progress and Directions. In *Companion Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) (WWW '25). Association for Computing Machinery, New York, NY, USA, 13–16.
- [8] Jian Jia, Jingtong Gao, Ben Xue, Junhao Wang, Qingpeng Cai, Quan Chen, Xiangyu Zhao, Peng Jiang, and Kun Gai. 2025. From principles to applications: A comprehensive survey of discrete tokenizers in generation, comprehension, recommendation, and information retrieval. *arXiv preprint arXiv:2502.12448* (2025).
- [9] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvish Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner's Handbook. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management* (Seoul, Republic of Korea) (CIKM '25). Association for Computing Machinery, New York, NY, USA, 6420–6425.
- [10] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.
- [11] Karl Krauth, Yixin Wang, and Michael Jordan. 2025. Breaking Feedback Loops in Recommender Systems with Causal Inference. *ACM Trans. Recomm. Syst.* 4, 1, Article 14 (July 2025), 20 pages.
- [12] Johannes Antonius La Poutré. 1989. *New techniques for the union-find problem*. Department of Computer Science, Utrecht University.
- [13] AJ Lawrance and PAW Lewis. 1977. An exponential moving-average sequence and point process (EMA1). *Journal of Applied Probability* 14, 1 (1977), 98–113.
- [14] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024).
- [15] Chang Liu, Yimeng Bai, Xiaoyan Zhao, Yang Zhang, Fuli Feng, and Wenge Rong. 2025. DiscRec: Disentangled Semantic-Collaborative Modeling for Generative Recommendation. *arXiv preprint arXiv:2506.15576* (2025).
- [16] Chang Liu, Yimeng Bai, Xiaoyan Zhao, Yang Zhang, Qifan Wang, Fuli Feng, and Wenge Rong. 2026. Intuition-Guided Latent Reasoning for LLM-Based Recommendation. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Republic of Korea) (KDD '26). Association for Computing Machinery, New York, NY, USA, 3140–3151.
- [17] Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. 2025. Generative Recommender with End-to-End Learnable Item Tokenization. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Padua, Italy) (SIGIR '25). Association for Computing Machinery, New York, NY, USA, 729–739.
- [18] Xiaoyu Liu, Fuwei Zhang, Yiqing Wu, Xinyu Jia, Zenghua Xia, Fuzhen Zhuang, Zhao Zhang, Fei Jiang, and Wei Lin. 2026. CAT-ID2: Category-Tree Integrated Document Identifier Learning for Generative Retrieval In E-commerce. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining* (USA) (WSDM '26). Association for Computing Machinery, New York, NY, USA, 426–435.
- [19] Yuri Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.
- [20] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.
- [21] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Keshavan, Trung Vu, Lukasz Heidt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender systems with generative retrieval. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., Red Hook, NY, USA.
- [22] Piyush Sao, Andrey Prokopenko, and Damien Lebrun-Grandie. 2024. PANDORA: A Parallel Dendrogram Construction Algorithm for Single Linkage Clustering on GPU. In *Proceedings of the 53rd International Conference on Parallel Processing* (Gotland, Sweden) (ICPP '24). Association for Computing Machinery, New York, NY, USA, 908–918.
- [23] Zihua Si, Zhongxiang Sun, Jiale Chen, Guozhang Chen, Xiaoxue Zang, Kai Zheng, Yang Song, Xiao Zhang, Jun Xu, and Kun Gai. 2024. Generative Retrieval with Semantic Tree-Structured Identifiers and Contrastive Learning. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region* (Tokyo, Japan) (SIGIR-AP 2024). Association for Computing Machinery, New York, NY, USA, 154–163.
- [24] Aaron van den Oord, Oriol Vinyals, and koray kavukcuoglu. 2017. Neural Discrete Representation Learning. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc., Red Hook, NY, USA.
- [25] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable Item Tokenization for Generative Recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management* (Boise, ID, USA) (CIKM '24). Association for Computing Machinery, New York, NY, USA, 2400–2409.
- [26] Yi Xu, Moyu Zhang, Chaofan Fan, Jinxin Hu, Xiaochen Li, Yu Zhang, Xiaoyi Zeng, and Jing Zhang. 2025. MMQ-v2: Align, Denoise, and Amplify: Adaptive Behavior Mining for Semantic IDs Learning in Recommendation. *arXiv preprint arXiv:2510.25622* (2025).
- [27] Jing Yan, Liu Jiang, Jianfei Cui, Zhichen Zhao, Xingyan Bin, Feng Zhang, and Zuotao Liu. 2024. Trinity: Syncrctizing Multi-/Long-Tail/Long-Term Interests All in One. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) (KDD '24). Association for Computing Machinery, New York, NY, USA, 6095–6104.
- [28] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
- [29] Yuhao Yang, Zhi Ji, Zhaopeng Li, Yi Li, Zhonglin Mo, Yue Ding, Kai Chen, Zijian Zhang, Jie Li, Shuanglong Li, and Lin Liu. 2025. Sparse Meets Dense: Unified Generative Recommendations with Cascaded Sparse-Dense Representations. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., Red Hook, NY, USA.
- [30] Zhengyi Yang, Xiangnan He, Jizhi Zhang, Jiancan Wu, Xin Xin, Jiawei Chen, and Xiang Wang. 2023. A Generic Learning Framework for Sequential Recommendation with Distribution Shifts. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Taipei, Taiwan) (SIGIR '23). Association for Computing Machinery, New York, NY, USA, 331–340.
- [31] Yang Zhang, Wenxin Xu, Xiaoyan Zhao, Wenjie Wang, Fuli Feng, Xiangnan He, and Tat-Seng Chua. 2026. Reinforced Latent Reasoning for LLM-based Recommendation. In *International Conference on Learning Representations*.
- [32] Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, et al. 2025. OneRec-v2 Technical Report. *arXiv preprint arXiv:2508.20900* (2025).
- [33] Han Zhu, Daqing Chang, Ziru Xu, Pengye Zhang, Xiang Li, Jie He, Han Li, Jian Xu, and Kun Gai. 2019. Joint optimization of tree-based index and deep model for recommender systems. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc., Red Hook, NY, USA.

- [34] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning Tree-based Deep Model for Recommender Systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (London, United Kingdom) (*KDD '18*). Association for Computing Machinery, New York, NY, USA, 1079–1088.
- [35] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, Huizhi Yang, Zheng Chai, Zhe Chen, Yuchao Zheng, Qiwei Chen, Feng Zhang, Xun Zhou, Peng Xu, Xiao Yang, Di Wu, and Zuotao Liu. 2025. RankMixer: Scaling Up Ranking Models in Industrial Recommenders. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management* (Seoul, Republic of Korea) (*CIKM '25*). Association for Computing Machinery, New York, NY, USA, 6309–6316.
- [36] Jingwei Zhuo, Ziru Xu, Wei Dai, Han Zhu, Han Li, Jian Xu, and Kun Gai. 2020. Learning optimal tree models under beam search. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 11650–11659.